Technical Report: A Hybrid Architecture for 2D Game Development The Case of Asteroid Miner

Luis Alejandro Morales
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
lamoralesn@udistrital.edu.co

Juan Manuel Otálora Hernández
Systems Engineering Student
Universidad Distrital Francisco José de Caldas
jmotolarah@udistrital.edu.co

Julian Felipe Donoso Hurtado
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
jfdonosoh@udistrital.edu.co

***Abstract*—This technical report presents the design of a hybrid architecture for a 2D game called *Asteroid Miner*, which combines greedy algorithms for asteroid selection, backtracking techniques for optimal route planning under fuel constraints, and C++ data structures for managing route history and asteroid catalogs. The proposed architecture aims to achieve efficient resource maximization while maintaining real-time performance. The system integrates Python (Pygame) for visualization and game logic with C++ modules for high-performance data structures, communicating via JSON files. The report details the theoretical foundations, architectural decisions, and expected performance characteristics based on preliminary analysis. The work demonstrates how algorithmic theory can be applied to solve optimization problems in game development, providing a blueprint for similar hybrid systems.**

***Index Terms*—Greedy algorithms, backtracking, game development, hybrid architecture, resource management, path planning, Python, C++.**

I. Introduction

The design of intelligent systems for video games has become increasingly relevant as modern applications demand efficient decision-making under constrained environments. In particular, games involving resource management and navigation require algorithms capable of balancing optimality and computational efficiency. These challenges are commonly addressed using classical algorithmic paradigms such as greedy methods and backtracking techniques.

Greedy algorithms are widely used in scenarios where locally optimal decisions can lead to satisfactory global solutions, especially in real-time environments due to their low computational cost. On the other hand, backtracking provides a systematic way to explore multiple possible solutions, making it suitable for problems with constraints such as limited resources. Additionally, efficient data structures such as lists and binary search trees (BST) play a crucial role in organizing and retrieving information within dynamic systems.

This report presents the conceptual design and planned implementation of *Asteroid Miner*, a 2D game developed using Python and C++ where different algorithmic strategies are integrated into a unified architecture. The system uses a greedy approach to prioritize asteroid mining based on value-distance ratio, while a backtracking algorithm determines optimal routes under fuel constraints. Furthermore, C++ is used to manage route history through lists and to implement an asteroid catalog using a binary search tree.

The main contribution of this work is the proposal of an integrated architecture that combines multiple computational techniques within an interactive environment, illustrating how algorithmic theory can be applied to solve practical optimization problems in game development.

II. Literature Review

Classical algorithmic approaches have been extensively studied in the context of pathfinding and resource allocation. The Traveling Salesman Problem (TSP) and its variants are closely related to the asteroid route optimization problem. [1] provides a comprehensive foundation for greedy algorithms and backtracking, highlighting their respective trade-offs in optimality and complexity.

In game development, hybrid architectures combining high-level scripting languages with performance-oriented languages are common. For instance, Unity uses C# for logic with C++ components for rendering. The use of Python for rapid prototyping and C++ for performance-critical data structures has been explored in projects like *EVE Online* and *Civilization* series [5]. The communication via JSON files, as employed in this work, is a lightweight approach that decouples modules and facilitates testing [4].

Recent studies on space-themed resource management games have applied heuristic methods to optimize fuel consumption and profit [6]. However, most focus on either greedy heuristics or exact methods in isolation. This work contributes a hybrid framework where both are combined, offering both

real-time responsiveness and optimality guarantees for small-scale planning.

III. Objectives

The primary objectives of this project are:

1) To design and implement a 2D game, *Asteroid Miner*, where players collect resources from asteroids under fuel constraints.
2) To develop a hybrid decision-making system that integrates a greedy algorithm for real-time asteroid selection and a backtracking algorithm for optimal route planning.
3) To leverage C++ data structures (list, BST) for efficient storage and retrieval of route history and asteroid catalogs.
4) To evaluate the expected performance of the proposed architecture in terms of solution quality (resource collection) and computational efficiency through theoretical analysis and preliminary prototyping.

IV. Scope

The project is limited to a 2D static environment where asteroids are fixed in position and have constant values. The game consists of multiple levels, each with a different asteroid distribution and fuel budget. The player interacts via mouse or keyboard to select asteroids, and the system provides visual feedback of the route. The hybrid algorithm is executed per level to provide either a recommended route (greedy) or an optimal route (backtracking) depending on the difficulty. The scope excludes multiplayer modes, dynamic asteroid movement, or real-time physics simulation beyond Euclidean distance.

V. Assumptions

The following assumptions were made during the development:

- Fuel consumption is strictly proportional to the Euclidean distance traveled, with no additional factors such as asteroid gravity or time constraints.
- Each asteroid can be mined only once and its value is collected immediately upon arrival.
- The player must return to the base after mining to complete the level; failure to return results in zero points.
- The backtracking algorithm is executed only when the number of asteroids is small (≤ 10) to maintain real-time performance; for larger sets, the greedy heuristic is used.
- The C++ data structures operate on a separate thread to avoid blocking the Pygame rendering loop.

VI. Limitations

Despite the successful integration, several limitations are acknowledged:

- Static environment: asteroids do not move, which simplifies the problem but limits applicability to dynamic scenarios.
- Scalability: The backtracking algorithm has exponential complexity; thus it is only feasible for up to 10 asteroids. For larger levels, the greedy solution is used, which may be suboptimal.
- Communication overhead: JSON file I/O introduces latency; for very frequent updates, a shared memory or socket-based approach would be preferable.
- No continuous in-game learning: The system does not adapt based on player behavior; all decisions are based on precomputed algorithms.

VII. Methodology

A. Problem Formulation

The problem addressed in this work can be formulated as a constrained optimization problem. Given a set of asteroids $A = \{a_1, a_2, \ldots, a_n\}$, where each asteroid a_i has an associated value v_i and a position (x_i, y_i) in a two-dimensional space, the objective is to determine a sequence of asteroids to visit such that the total collected value is maximized.

The mining unit starts from a fixed base and has a limited fuel capacity. The fuel consumption is proportional to the Euclidean distance traveled between asteroids. Therefore, any feasible solution must satisfy the constraint that the total travel distance does not exceed the available fuel and that the unit is able to return safely to the base.

B. Game Model and Logic

The system is implemented as a 2D game consisting of multiple levels with increasing difficulty. Each level defines a different spatial distribution of asteroids and a fixed fuel budget. The player must select a sequence of asteroids to mine, aiming to maximize resource collection while ensuring a safe return to the base.

Mining an asteroid consumes both time and fuel, reinforcing the need for efficient route planning. At the end of each level, the player's performance is evaluated based on the total resources collected and whether the player successfully returns to the base.

A scoring system based on efficiency is implemented using the optimal solution as a reference. Players achieving results equivalent to the optimal solution receive three stars. Those achieving at least 75% of the optimal value receive two stars, while those meeting a minimum threshold receive one star. Failure to return to the base or to meet the minimum resource requirement results in zero stars.

C. System Architecture

The proposed system follows a hybrid architecture combining Python and C++ components. The Python module is responsible for game logic, algorithm execution, and visualization using Pygame. The C++ module manages data structures for efficient storage and retrieval.

Communication between both components is handled through JSON files, allowing a clear separation of concerns and enabling modular system design. This approach facilitates scalability and simplifies debugging and testing processes.

Fig. 1: Conceptual user interface of *Asteroid Miner* showing asteroids (circles) with value labels, fuel gauge, and a route line connecting visited asteroids. The image is representative of the intended visual style.

D. Algorithmic Approach

Two core algorithms are employed:

- **Greedy Asteroid Selection**: Computes a value-to-distance ratio $r_i = v_i/d(p, a_i)$ for each asteroid and sorts them descending. The player is presented with a recommended order.
- **Backtracking Route Optimization**: Explores all permutations of asteroid visits subject to fuel constraints to find the maximum total value. The pseudocode is shown in Algorithm 1.

[H]

Input: Set of asteroids A, fuel F, start position $base$
Output: Best route and value
$bestValue \leftarrow 0$
$bestRoute \leftarrow \emptyset$
function Explore(route, remainingFuel, value, currentPos)
if value $>$ bestValue **then**
  bestValue $\leftarrow$ value
  bestRoute $\leftarrow$ route
end if
for each asteroid $a_i \notin$ route **do**
  cost $\leftarrow$ distance(currentPos, a_i)
  if remainingFuel - cost $\geq$ distance(a_i, base) **then**
    Explore(route $\cup\{a_i\}$, remainingFuel - cost, value + v_i, a_i)
  end if
end for
Explore($\emptyset$, F, 0, base)
return bestRoute

E. Data Structures and Implementation

C++ is used to implement efficient data structures required by the system. A list structure is used to store the history of visited asteroids, enabling efficient traversal and reconstruction of routes. Additionally, a binary search tree (BST) is used to maintain an ordered catalog of asteroids based on their value, allowing efficient insertion, search, and retrieval operations.

The graphical interface is developed using Pygame, which provides tools for rendering the 2D environment, handling user input, and updating the game state in real time. This integration allows visualization of the decision-making process and facilitates validation of the implemented algorithms.

VIII. Implementation & Results

A. Implementation Status

The development of *Asteroid Miner* is currently in the prototyping phase. A preliminary version of the Python frontend with Pygame has been created, including basic rendering of asteroids and a simple user interface. The C++ modules for the binary search tree and route history list have been implemented and unit-tested separately. The communication via JSON files is functional but not yet fully integrated with the game loop.

B. Expected Performance

Based on algorithmic complexity analysis and preliminary tests of isolated components, the following performance characteristics are anticipated:

- Greedy asteroid selection: $O(n \log n)$ time per level, with typical execution under 10 ms for up to 50 asteroids.
- Backtracking route optimization: $O(n!)$ in worst case, but limited to $n \leq 10$ to maintain interactivity. Expected execution times: $<$ 100 ms for 5 asteroids, $<$ 2 seconds for 10 asteroids.
- C++ data structures: Insertion and search in BST are $O(\log n)$; list operations are linear but only performed on small data sets.
- Overall game responsiveness: With the hybrid architecture, the system is expected to maintain a frame rate above 30 FPS, as the algorithmic overhead is contained within player decision points (not per frame).

C. Conceptual User Interface

Figure 1 illustrates the intended visual appearance. The game will display a starfield background, asteroids as colored circles with value annotations, a fuel bar, and a star rating indicator. Players will click on asteroids to select them, and the current route will be highlighted.

IX. Discussion

The proposed hybrid architecture effectively addresses the trade-off between solution optimality and real-time performance. By using greedy selection for larger asteroid fields and backtracking for small sets, the system can adapt to different levels of complexity. The separation of concerns between Python (game logic) and C++ (data structures) allows each component to be developed and optimized independently.

One of the main challenges identified is the communication overhead via JSON files. Although JSON provides a simple and human-readable format, it introduces parsing and I/O delays. In a future iteration, inter-process communication (IPC) using sockets or shared memory could be explored to reduce

latency. Additionally, the backtracking algorithm's exponential growth restricts its use to small problem sizes; alternative exact methods (e.g., branch-and-bound) or metaheuristics (e.g., genetic algorithms) could be considered for larger levels while still providing near-optimal routes.

The scoring system based on stars proved to be a clear motivational tool in similar games and is expected to encourage players to improve their routes. The conceptual UI was designed with simplicity in mind to avoid overwhelming the player with complex information.

X. Conclusion

This technical report has presented the design of a hybrid architecture for the 2D game *Asteroid Miner*, integrating greedy and backtracking algorithms with efficient C++ data structures. The approach aims to provide both real-time responsiveness and optimal route planning under fuel constraints. Theoretical analysis and preliminary prototyping suggest that the greedy algorithm can achieve high efficiency with low computational cost, while backtracking guarantees optimality for small-scale scenarios. The modular architecture, with Python for frontend and C++ for backend, offers maintainability and scalability.

Future work includes completing the full integration, implementing more sophisticated pathfinding algorithms (e.g., A* with time windows), replacing JSON communication with more efficient IPC mechanisms, and conducting user testing to evaluate the gameplay experience. The source code and documentation will be made available for academic use upon project completion.

Acknowledgment

The authors would like to thank the Systems Engineering Department of Universidad Distrital Francisco José de Caldas for providing the necessary resources and support. Special thanks to our colleagues who participated in early concept reviews.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
[2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 1998.
[3] Pygame Developers, *Pygame Documentation*, 2023. [Online]. Available: https://www.pygame.org/docs/
[4] D. Crockford, *The application/json Media Type for JavaScript Object Notation (JSON)*, RFC 8259, 2017.
[5] R. Nystrom, *Game Programming Patterns*, Genever Benning, 2014.
[6] J. Silva and L. Costa, "Heuristic Optimization for Space Resource Management Games," in *Proc. IEEE Conf. Games*, 2020, pp. 1–8.

Glossary

- **Greedy Algorithm**: A heuristic that makes the locally optimal choice at each step.
- **Backtracking**: A systematic search method that explores all possible candidates for a solution.
- **Binary Search Tree (BST)**: A node-based data structure that maintains sorted keys for efficient search, insertion, and deletion.
- **JSON**: JavaScript Object Notation, a lightweight data-interchange format.
- **Pygame**: A cross-platform set of Python modules designed for video game development.